

MOMO Quant

Part 4 — API Specification

Version: 1.0 Draft

Backend: .NET 8 Web API

Frontend: React Dashboard

Realtime: SignalR

API Style: REST + SignalR events

Primary Goal: Define clear backend contracts before implementation.

1. API Design Principles

The API must be:

- predictable
- versioned
- consistent
- easy for Cursor to implement
- easy for the React dashboard to consume
- safe by default
- auditable
- expandable

The API should not expose internal trading complexity directly to the frontend. The frontend sends commands and reads state. Trading decisions remain inside the backend.

2. Base API Format

All REST APIs should use this base path:

```
/api/v1
```

Example:

```
/api/v1/auth/login  
/api/v1/backtests/run  
/api/v1/trading-sessions
```

3. Standard Response Format

All successful API responses should use:

```
{
  "success": true,
  "message": "Request completed successfully.",
  "data": {}
}
```

All failed API responses should use:

```
{
  "success": false,
  "message": "Validation failed.",
  "errors": [
    {
      "field": "symbol",
      "message": "Symbol is required."
    }
  ]
}
```

4. Pagination Format

List endpoints should support:

```
page
pageSize
sortBy
sortDirection
search
```

Example:

```
GET /api/v1/trades?page=1&pageSize=50&sortBy=openedAtUtc&sortDirection=desc
```

Response:

```
{
  "success": true,
  "message": "Trades loaded.",
  "data": {
    "items": [],
    "page": 1,
    "pageSize": 50,
    "totalItems": 500,
    "totalPages": 10
  }
}
```

5. Authentication APIs

5.1 Login

`POST /api/v1/auth/login`

Request:

```
{
  "email": "admin@momoquant.local",
  "password": "password"
}
```

Response:

```
{
  "success": true,
  "message": "Login successful.",
  "data": {
    "accessToken": "jwt-token",
    "expiresAtUtc": "2026-01-01T00:00:00Z",
    "user": {
      "id": 1,
      "fullName": "Admin",
      "email": "admin@momoquant.local",
      "role": "Admin"
    }
  }
}
```

```
}  
}
```

5.2 Current User

```
GET /api/v1/auth/me
```

Returns the logged-in user profile.

5.3 Logout

```
POST /api/v1/auth/logout
```

For JWT-only MVP, this can simply return success. Later token blacklisting can be added.

6. User Management APIs

Admin-only.

6.1 Get Users

```
GET /api/v1/users
```

6.2 Get User By Id

```
GET /api/v1/users/{id}
```

6.3 Create User

```
POST /api/v1/users
```

Request:

```
{
  "fullName": "Trader User",
  "email": "trader@momoquant.local",
  "password": "StrongPassword123",
  "roleId": 2
}
```

6.4 Update User

```
PUT /api/v1/users/{id}
```

6.5 Disable User

```
POST /api/v1/users/{id}/disable
```

7. Exchange APIs

7.1 Get Exchanges

```
GET /api/v1/exchanges
```

7.2 Create Exchange

```
POST /api/v1/exchanges
```

Request:

```
{
  "name": "Binance Futures",
  "code": "BINANCE_FUTURES",
  "baseUrl": "https://fapi.binance.com",
}
```

```
"websocketUrl": "wss://fstream.binance.com"
}
```

7.3 Update Exchange

```
PUT /api/v1/exchanges/{id}
```

7.4 Test Exchange Connectivity

```
POST /api/v1/exchanges/{id}/test-connection
```

Response:

```
{
  "success": true,
  "message": "Exchange connection successful.",
  "data": {
    "restLatencyMs": 122,
    "websocketAvailable": true
  }
}
```

8. API Key Vault APIs

Admin-only.

8.1 Get API Key Profiles

```
GET /api/v1/api-keys
```

Must not return raw encrypted values.

8.2 Create API Key Profile

```
POST /api/v1/api-keys
```

Request:

```
{
  "exchangeId": 1,
  "name": "Binance Testnet Key",
  "apiKey": "raw-api-key",
  "apiSecret": "raw-api-secret",
  "passphrase": null,
  "isTestnet": true
}
```

Backend encrypts before storing.

8.3 Disable API Key Profile

```
POST /api/v1/api-keys/{id}/disable
```

9. Symbol APIs

9.1 Get Symbols

```
GET /api/v1/symbols
```

Optional filters:

```
exchangeId
isActive
search
```

9.2 Sync Symbols From Exchange

```
POST /api/v1/symbols/sync
```

Request:

```
{  
  "exchangeId": 1  
}
```

9.3 Update Symbol Status

```
PUT /api/v1/symbols/{id}/status
```

Request:

```
{  
  "isActive": true  
}
```

10. Market Data APIs

10.1 Get Candles

```
GET /api/v1/market-data/candles
```

Query:

```
symbolId  
timeframe  
fromUtc  
toUtc  
limit
```

Example:


```
GET /api/v1/market-data/candles?  
symbolId=1&timeframe=3m&fromUtc=2026-01-01T00:00:00Z&toUtc=2026-01-02T00:00:00Z
```

10.2 Import Historical Candles

```
POST /api/v1/market-data/candles/import
```

Request:

```
{  
  "exchangeId": 1,  
  "symbolId": 1,  
  "timeframe": "3m",  
  "fromUtc": "2026-01-01T00:00:00Z",  
  "toUtc": "2026-01-31T23:59:59Z"  
}
```

10.3 Get Import Status

```
GET /api/v1/market-data/imports/{importId}
```

10.4 Get Latest Market Snapshot

```
GET /api/v1/market-data/snapshot
```

Query:

```
symbolId  
timeframe
```

Returns latest candle, indicators, spread if available, and bot status for that symbol.

11. Indicator APIs

11.1 Get Indicator Snapshot

```
GET /api/v1/indicators/snapshot
```

Query:

```
symbolId  
timeframe  
candleId
```

11.2 Recalculate Indicators

```
POST /api/v1/indicators/recalculate
```

Request:

```
{  
  "symbolId": 1,  
  "timeframe": "3m",  
  "fromUtc": "2026-01-01T00:00:00Z",  
  "toUtc": "2026-01-31T23:59:59Z"  
}
```

12. Strategy APIs

12.1 Get Strategies

```
GET /api/v1/strategies
```

12.2 Get Strategy Details

```
GET /api/v1/strategies/{id}
```

12.3 Enable Strategy

```
POST /api/v1/strategies/{id}/enable
```

12.4 Disable Strategy

```
POST /api/v1/strategies/{id}/disable
```

12.5 Get Strategy Parameters

```
GET /api/v1/strategies/{id}/parameters
```

12.6 Update Strategy Parameters

```
PUT /api/v1/strategies/{id}/parameters
```

Request:

```
{
  "parameters": [
    {
      "key": "EmaFastPeriod",
      "value": "20",
      "valueType": "int",
      "timeframe": "3m",
      "symbolId": null
    },
    {
      "key": "AtrMultiplier",
```

```
    "value": "1.5",
    "valueType": "decimal",
    "timeframe": "3m",
    "symbolId": null
  }
]
```

13. Trading Session APIs

Trading sessions are used for backtests, replay, paper trading, and live trading.

13.1 Get Trading Sessions

```
GET /api/v1/trading-sessions
```

Filters:

```
mode
status
fromUtc
toUtc
```

13.2 Get Trading Session Details

```
GET /api/v1/trading-sessions/{id}
```

13.3 Stop Trading Session

```
POST /api/v1/trading-sessions/{id}/stop
```

13.4 Pause Trading Session

```
POST /api/v1/trading-sessions/{id}/pause
```

13.5 Resume Trading Session

```
POST /api/v1/trading-sessions/{id}/resume
```

14. Bot Control APIs

These APIs control the running bot state.

14.1 Get Bot Status

```
GET /api/v1/bot/status
```

Response:

```
{
  "success": true,
  "message": "Bot status loaded.",
  "data": {
    "mode": "Paper",
    "status": "Stopped",
    "emergencyStopEnabled": false,
    "activeSymbols": ["BTCUSDT", "ETHUSDT"],
    "activeStrategies": ["EMA_PULLBACK", "LIQUIDITY_SWEEP"],
    "lastMarketUpdateUtc": "2026-01-01T12:00:00Z"
  }
}
```

14.2 Change Trading Mode

```
POST /api/v1/bot/change-mode
```

Request:

```
{
  "mode": "Paper",
  "confirmationText": "CHANGE MODE"
}
```

Live mode requires stricter confirmation.

14.3 Start Bot

```
POST /api/v1/bot/start
```

Request:

```
{
  "mode": "Paper",
  "exchangeId": 1,
  "symbolIds": [1, 2, 3, 4, 5],
  "timeframe": "3m",
  "higherTimeframe": "15m",
  "strategyIds": [1, 2, 3],
  "riskProfileId": 1,
  "initialBalance": 10000
}
```

14.4 Stop Bot

```
POST /api/v1/bot/stop
```

14.5 Emergency Stop

```
POST /api/v1/bot/emergency-stop
```

This immediately stops new trade entries and attempts to cancel open orders.

14.6 Clear Emergency Stop

```
POST /api/v1/bot/clear-emergency-stop
```

Admin-only.

15. Backtesting APIs

15.1 Run Backtest

```
POST /api/v1/backtests/run
```

Request:

```
{
  "name": "BTC EMA Pullback January Test",
  "exchangeId": 1,
  "symbolIds": [1],
  "timeframe": "3m",
  "higherTimeframe": "15m",
  "fromUtc": "2026-01-01T00:00:00Z",
  "toUtc": "2026-01-31T23:59:59Z",
  "initialBalance": 10000,
  "strategyIds": [1, 2, 3],
  "riskProfileId": 1,
  "executionSettings": {
    "makerFeeRate": 0.0002,
    "takerFeeRate": 0.0005,
    "slippagePercent": 0.01,
    "simulateMakerFills": true,
    "orderTimeoutSeconds": 15
  }
}
```

Response:

```
{
  "success": true,
  "message": "Backtest started.",
  "data": {
```

```
"tradingSessionId": 101,  
"backtestRunId": 55,  
"status": "Running"  
}  
}
```

15.2 Get Backtest Run

```
GET /api/v1/backtests/{id}
```

15.3 Get Backtest Results

```
GET /api/v1/backtests/{id}/results
```

15.4 Get Backtest Trades

```
GET /api/v1/backtests/{id}/trades
```

15.5 Get Backtest Signals

```
GET /api/v1/backtests/{id}/signals
```

15.6 Cancel Backtest

```
POST /api/v1/backtests/{id}/cancel
```

16. Replay APIs

16.1 Create Replay Session

```
POST /api/v1/replay/create
```

Request:

```
{
  "name": "BTC Replay January 14",
  "exchangeId": 1,
  "symbolId": 1,
  "timeframe": "3m",
  "higherTimeframe": "15m",
  "fromUtc": "2026-01-14T00:00:00Z",
  "toUtc": "2026-01-14T23:59:59Z",
  "strategyIds": [1, 2, 3],
  "riskProfileId": 1,
  "initialBalance": 10000
}
```

16.2 Get Replay Session

```
GET /api/v1/replay/{id}
```

16.3 Start Replay

```
POST /api/v1/replay/{id}/start
```

16.4 Pause Replay

```
POST /api/v1/replay/{id}/pause
```

16.5 Resume Replay

```
POST /api/v1/replay/{id}/resume
```

16.6 Change Replay Speed

```
POST /api/v1/replay/{id}/speed
```

Request:

```
{
  "speed": 10
}
```

Allowed values:

```
1
2
10
50
100
```

16.7 Step Replay Forward

```
POST /api/v1/replay/{id}/step
```

Moves replay forward by one candle.

16.8 Stop Replay

```
POST /api/v1/replay/{id}/stop
```

17. Paper Trading APIs

17.1 Get Paper Accounts

```
GET /api/v1/paper/accounts
```

17.2 Create Paper Account

```
POST /api/v1/paper/accounts
```

Request:

```
{
  "name": "Default Paper Account",
  "initialBalance": 10000,
  "currency": "USDT"
}
```

17.3 Start Paper Trading

```
POST /api/v1/paper/start
```

Request:

```
{
  "paperAccountId": 1,
  "exchangeId": 1,
  "symbolIds": [1, 2, 3, 4, 5],
  "timeframe": "3m",
  "higherTimeframe": "15m",
  "strategyIds": [1, 2, 3],
  "riskProfileId": 1
}
```

17.4 Stop Paper Trading

```
POST /api/v1/paper/stop
```

17.5 Get Paper Account Snapshot

```
GET /api/v1/paper/accounts/{id}/snapshot
```

17.6 Get Paper Trades

```
GET /api/v1/paper/accounts/{id}/trades
```

18. Live Trading APIs

Live trading should be implemented later.

18.1 Get Live Trading Readiness

```
GET /api/v1/live/readiness
```

Response:

```
{
  "success": true,
  "message": "Live readiness checked.",
  "data": {
    "isReady": false,
    "checks": [
      {
        "name": "Exchange API Key",
        "passed": true
      },
      {
        "name": "Risk Profile Enabled",
        "passed": true
      }
    ]
  }
}
```

```
    },  
    {  
      "name": "Paper Trading Validation",  
      "passed": false  
    }  
  ]  
}  
}
```

18.2 Enable Live Trading

`POST /api/v1/live/enable`

Request:

```
{  
  "confirmationText": "ENABLE LIVE TRADING",  
  "apiKeyProfileId": 1,  
  "riskProfileId": 1  
}
```

This endpoint must be admin-only and heavily audited.

18.3 Disable Live Trading

`POST /api/v1/live/disable`

19. Signal APIs

19.1 Get Signals

`GET /api/v1/signals`

Filters:

```
tradingSessionId
symbolId
strategyId
signalType
direction
fromUtc
toUtc
```

19.2 Get Signal Details

```
GET /api/v1/signals/{id}
```

Should include related AI decision, risk decision, order, and trade if available.

20. AI APIs

These are dashboard-facing AI data APIs.

The backend-to-Python AI service endpoints are separate internal calls.

20.1 Get AI Decisions

```
GET /api/v1/ai/decisions
```

Filters:

```
tradingSessionId
symbolId
marketRegime
tradeAllowed
fromUtc
toUtc
```

20.2 Get AI Decision Details

```
GET /api/v1/ai/decisions/{id}
```

20.3 Run AI Market Regime Test

```
POST /api/v1/ai/regime/test
```

Request:

```
{
  "symbolId": 1,
  "timeframe": "3m",
  "fromUtc": "2026-01-01T00:00:00Z",
  "toUtc": "2026-01-02T00:00:00Z"
}
```

20.4 Get AI Performance Summary

```
GET /api/v1/ai/performance
```

Query:

```
tradingSessionId
fromUtc
toUtc
```

Should compare AI confidence scores with actual trade outcomes.

21. Risk APIs

21.1 Get Risk Profiles

```
GET /api/v1/risk/profiles
```

21.2 Create Risk Profile

```
POST /api/v1/risk/profiles
```

Request:

```
{
  "name": "Conservative",
  "description": "Low-risk paper trading profile."
}
```

21.3 Get Risk Rules

```
GET /api/v1/risk/profiles/{id}/rules
```

21.4 Update Risk Rules

```
PUT /api/v1/risk/profiles/{id}/rules
```

Request:

```
{
  "rules": [
    {
      "ruleKey": "MaxRiskPerTradePercent",
      "ruleValue": "0.5",
      "valueType": "decimal",
      "isEnabled": true
    },
    {
      "ruleKey": "MaxDailyLossPercent",
      "ruleValue": "2",
      "valueType": "decimal",
      "isEnabled": true
    },
    {
      "ruleKey": "MaxConsecutiveLosses",
      "ruleValue": "3",

```



```
    "valueType": "int",  
    "isEnabled": true  
  }  
]  
}
```

21.5 Get Risk Decisions

```
GET /api/v1/risk/decisions
```

Filters:

```
tradingSessionId  
symbolId  
decision  
fromUtc  
toUtc
```

22. Order APIs

22.1 Get Orders

```
GET /api/v1/orders
```

Filters:

```
tradingSessionId  
symbolId  
status  
mode  
fromUtc  
toUtc
```

22.2 Get Order Details

```
GET /api/v1/orders/{id}
```

22.3 Cancel Order

```
POST /api/v1/orders/{id}/cancel
```

For paper/live modes only.

23. Trade APIs

23.1 Get Trades

```
GET /api/v1/trades
```

Filters:

```
tradingSessionId  
symbolId  
strategyId  
status  
direction  
fromUtc  
toUtc
```

23.2 Get Trade Details

```
GET /api/v1/trades/{id}
```

Should include:

- entry order
- exit order
- fills

- signal
 - AI decision
 - risk decision
 - PnL breakdown
-

23.3 Manually Close Trade

```
POST /api/v1/trades/{id}/close
```

Paper/live modes only.

Request:

```
{  
  "reason": "ManualClose"  
}
```

24. Position APIs

24.1 Get Positions

```
GET /api/v1/positions
```

Filters:

```
tradingSessionId  
symbolId  
status
```

24.2 Get Position Details

```
GET /api/v1/positions/{id}
```

25. Reporting APIs

25.1 Get Dashboard Summary

```
GET /api/v1/reports/dashboard-summary
```

Query:

```
tradingSessionId  
fromUtc  
toUtc
```

Returns:

- balance
 - equity
 - PnL
 - win rate
 - open positions
 - active strategies
 - recent trades
 - current bot status
-

25.2 Get Performance Report

```
GET /api/v1/reports/performance
```

Query:

```
tradingSessionId  
fromUtc  
toUtc
```

Returns:

- total trades
- win rate
- net PnL
- max drawdown
- profit factor

- expectancy
 - average win
 - average loss
 - fees
 - funding fees
-

25.3 Get Strategy Report

```
GET /api/v1/reports/strategies
```

Query:

```
tradingSessionId  
fromUtc  
toUtc
```

25.4 Get Symbol Report

```
GET /api/v1/reports/symbols
```

25.5 Get Market Regime Report

```
GET /api/v1/reports/market-regimes
```

25.6 Get Rejection Report

```
GET /api/v1/reports/rejections
```

Shows why trades were rejected.

25.7 Get Missed Trade Report

```
GET /api/v1/reports/missed-trades
```

Shows valid signals that failed to fill because of maker-limit execution.

26. Monitoring APIs

26.1 Get System Health

```
GET /api/v1/monitoring/health
```

Response:

```
{
  "success": true,
  "message": "System health loaded.",
  "data": {
    "status": "Healthy",
    "services": [
      {
        "name": "API",
        "status": "Healthy",
        "latencyMs": 12
      },
      {
        "name": "Database",
        "status": "Healthy",
        "latencyMs": 25
      },
      {
        "name": "Redis",
        "status": "Healthy",
        "latencyMs": 5
      },
      {
        "name": "AI Service",
        "status": "Warning",
        "latencyMs": 280
      }
    ]
  }
}
```

```
}  
}
```

26.2 Get Exchange Health

```
GET /api/v1/monitoring/exchange-health
```

26.3 Get Worker Status

```
GET /api/v1/monitoring/workers
```

26.4 Get Error Logs

```
GET /api/v1/monitoring/errors
```

27. Audit Log APIs

Admin-only.

27.1 Get Audit Logs

```
GET /api/v1/audit-logs
```

Filters:

```
userId  
action  
entityType  
fromUtc  
toUtc
```

27.2 Get Audit Log Details

```
GET /api/v1/audit-logs/{id}
```

28. Settings APIs

28.1 Get App Settings

```
GET /api/v1/settings
```

28.2 Update App Setting

```
PUT /api/v1/settings/{key}
```

Request:

```
{  
  "value": "Paper"  
}
```

Sensitive settings require admin-only access.

29. Notification APIs

Optional for MVP but useful.

29.1 Get Notifications

```
GET /api/v1/notifications
```

29.2 Mark Notification As Read

```
POST /api/v1/notifications/{id}/read
```

30. SignalR Hubs

Use SignalR for real-time dashboard updates.

30.1 Bot Hub

```
/hubs/bot
```

Events sent to frontend:

```
BotStatusChanged  
TradingModeChanged  
EmergencyStopTriggered  
EmergencyStopCleared
```

30.2 Market Hub

```
/hubs/market
```

Events:

```
CandleUpdated  
IndicatorUpdated  
MarketSnapshotUpdated
```

30.3 Trading Hub

```
/hubs/trading
```

Events:

```
SignalGenerated  
AiDecisionCreated  
RiskDecisionCreated  
OrderCreated  
OrderUpdated  
OrderFilled  
TradeOpened  
TradeClosed  
PositionUpdated  
MissedOrderCreated
```

30.4 Replay Hub

```
/hubs/replay
```

Events:

```
ReplayStarted  
ReplayPaused  
ReplayStopped  
ReplayTick  
ReplayDecisionUpdated  
ReplayTradeUpdated
```

30.5 Monitoring Hub

```
/hubs/monitoring
```

Events:

```
HealthStatusChanged  
ExchangeConnectionChanged  
WorkerStatusChanged  
ErrorLogged
```

31. Internal Python AI Service API

These endpoints are internal between .NET and Python.

Base URL:

```
http://momo-ai:8000
```

31.1 Detect Market Regime

```
POST /ai/regime/detect
```

Request:

```
{
  "symbol": "BTCUSDT",
  "timeframe": "3m",
  "candles": [],
  "indicators": {}
}
```

Response:

```
{
  "marketRegime": "Trending",
  "confidence": 82,
  "explanation": "EMA alignment and directional candles indicate bullish trend."
}
```

31.2 Score Trade Confidence

```
POST /ai/confidence/score
```

Request:

```
{
  "symbol": "BTCUSDT",
  "timeframe": "3m",
  "marketRegime": "Trending",
  "strategySignals": [],
  "indicators": {},
  "riskContext": {}
}
```

Response:

```
{
  "confidenceScore": 86,
  "tradeAllowed": true,
  "preferredStrategyCode": "EMA_PULLBACK",
  "riskAdjustment": "Normal",
  "explanation": "Trend, volume, and pullback conditions align."
}
```

31.3 Optimize Parameters

```
POST /ai/parameters/optimize
```

Request:

```
{
  "strategyCode": "EMA_PULLBACK",
  "symbol": "BTCUSDT",
  "timeframe": "3m",
  "historicalPerformance": {},
  "parameterBounds": {}
}
```

31.4 Detect Anomaly

```
POST /ai/anomaly/detect
```

Request:

```
{
  "symbol": "BTCUSDT",
  "timeframe": "3m",
  "latestCandles": [],
  "exchangeHealth": {},
  "spread": 0.02
}
```

Response:

```
{
  "isAnomaly": false,
  "severity": "Low",
  "reason": "Market activity is within normal range."
}
```

31.5 Explain Trade

```
POST /ai/trade/explain
```

Request:

```
{
  "signal": {},
  "aiDecision": {},
  "riskDecision": {},
  "executionDecision": {}
}
```

Response:

```
{
  "explanation": "Trade was approved because EMA trend was bullish, liquidity sweep confirmed, confidence was 88, and risk limits were valid."
}
```

32. Authorization Rules

Admin

Can access everything.

Trader

Can:

- run backtests
- run replay
- start/stop paper trading
- view reports
- view trades
- view orders
- update strategy parameters if allowed
- view monitoring

Cannot:

- manage users
- enable live trading
- manage API keys unless explicitly allowed

Viewer

Can only read:

- dashboard
- reports
- trades
- positions
- monitoring

Cannot start, stop, update, or trade.

33. API Implementation Order

Do not build all APIs at once.

Correct order:

1. Auth APIs
2. User APIs
3. Exchange APIs
4. Symbol APIs
5. Market Data APIs
6. Strategy APIs
7. Risk APIs
8. Trading Session APIs
9. Backtesting APIs
10. Replay APIs
11. Paper Trading APIs
12. Signal / AI / Risk Decision APIs
13. Order / Trade / Position APIs
14. Reporting APIs
15. Monitoring APIs
16. Settings / Audit APIs
17. Live Trading APIs later

34. MVP API Scope

Build these first:

Auth
Users
Exchanges
Symbols
Market Data
Strategies
Risk Profiles
Trading Sessions
Backtesting
Replay
Paper Trading
Signals
Trades
Orders
Reports
Monitoring
Settings
Audit Logs

Delay:

Live Trading

Notifications

Advanced AI optimization

Multi-exchange routing

Order book APIs

Funding reports

35. Critical API Rules

1. No frontend endpoint should place trades directly.
2. All trading actions go through backend command handlers.
3. Live trading APIs must be disabled until explicitly implemented.
4. Every state-changing endpoint must create an audit log.
5. All timestamps must be UTC.
6. All financial values must use decimal.
7. Every API must return the standard response format.
8. All list APIs must support pagination.
9. Every bot control action must be permission checked.
10. Emergency stop must be fast and reliable.
11. API keys must never be returned in raw form.
12. Backtesting, replay, paper, and live should share common trading abstractions.

36. Final API Summary

The MOMO Quant API should provide a clean control layer for the dashboard while keeping trading intelligence inside the backend.

The most important design rule is:

The frontend controls and observes the system.

The backend decides, validates, executes, logs, and protects the system.

This separation prevents the dashboard from becoming a dangerous trading brain.